

Chapter 1: Multiple Linear Regression in R

Syntax Notes, Mathematical Connections, and Ten Guided Exercises

STAT452 – Applied Statistics for Engineers and Scientists II

Contents

1	What this worksheet covers	1
2	R syntax you should know	2
2.1	The basic mental model	2
2.2	Common syntax patterns	2
2.3	Regression formula syntax	2
3	Translation between mathematics and R	3
3.1	Matrix operations in R	3
3.2	How to read <code>summary(lm(...))</code>	3
4	Exercises	4
5	Worked solutions	6
	One-page R regression checklist	10

1 What this worksheet covers

This worksheet follows Chapter 1 of the course: multiple linear regression, least squares, fitted values, residuals, coefficient inference, R^2 , nested model comparison, and residual diagnostics. All examples use `mtcars`, a data frame included with R, so no package or download is required.

We will mainly use these variables:

R name	Meaning	Unit
<code>mpg</code>	fuel efficiency	miles per US gallon
<code>wt</code>	vehicle weight	1000 pounds
<code>hp</code>	gross horsepower	horsepower

The main model will be

$$Y_i = \beta_0 + \beta_1 x_{i,\text{wt}} + \beta_2 x_{i,\text{hp}} + \varepsilon_i,$$

or, in matrix form,

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

2 R syntax you should know

2.1 The basic mental model

R stores information in *objects*. You create an object with `<-`, then give that object to functions that calculate a result.

```
x <- c(2, 4, 6, 8)  # Store four values in x
mean(x)            # Give x to the mean() function
sd(x)              # Compute its sample standard deviation
```

The symbol `#` starts a comment. R ignores everything after it on the same line.

2.2 Common syntax patterns

Syntax	Meaning
<code>x <- 5</code>	Assign the value 5 to the name <code>x</code> .
<code>c(2, 4, 6)</code>	Combine values into a vector.
<code>cars\$wt</code>	Select the <code>wt</code> column from the data frame <code>cars</code> .
<code>cars[1:5,]</code>	Select rows 1 through 5 and every column.
<code>cars[, c("mpg", "wt")]</code>	Select every row and two named columns.
<code>mean(x)</code>	Call a function; <code>x</code> is its argument.
<code>sum(x²)</code>	Square every element of <code>x</code> , then add the results.
<code>sqrt(x)</code>	Take square roots element by element.
<code>length(x)</code>	Count the elements of a vector.
<code>nrow(cars)</code>	Count observations in a data frame.
<code>head(cars)</code>	Display the first six rows.
<code>str(cars)</code>	Display the structure and variable types.
<code>summary(cars)</code>	Produce quick summaries of every variable.

2.3 Regression formula syntax

The R formula

```
mpg ~ wt + hp
```

means “model `mpg` using an intercept, `wt`, and `hp`.” It corresponds to

$$\text{mpg}_i = \beta_0 + \beta_1 \text{wt}_i + \beta_2 \text{hp}_i + \varepsilon_i.$$

R formula	Meaning
<code>y ~ x</code>	intercept plus one predictor
<code>y ~ x1 + x2</code>	intercept plus two predictors
<code>y ~ .</code>	use every other column as a predictor
<code>y ~ x - 1</code>	omit the intercept
<code>y ~ x1:x2</code>	interaction only
<code>y ~ x1*x2</code>	both main effects and their interaction
<code>y ~ x + I(x²)</code>	linear and quadratic terms

Important: In a formula, `+` means “include another term.” It does not tell R to numerically add the two columns together.

3 Translation between mathematics and R

Statistical object	Mathematics	R
Fit the model	minimize $\sum e_i^2$	<code>fit <- lm(y ~ x1 + x2, data=d)</code>
Coefficient estimates	$\hat{\beta}$	<code>coef(fit)</code>
Design matrix	$\mathbf{X}$	<code>model.matrix(fit)</code>
Fitted values	$\hat{\mathbf{Y}} = \mathbf{X}\hat{\beta}$	<code>fitted(fit)</code>
Residuals	$\mathbf{e} = \mathbf{Y} - \hat{\mathbf{Y}}$	<code>resid(fit)</code>
Residual sum of squares	$\text{SSE} = \sum e_i^2$	<code>sum(resid(fit)^2)</code>
Error degrees of freedom	$n - p - 1$	<code>df.residual(fit)</code>
MSE	$\text{SSE}/(n - p - 1)$	<code>deviance(fit)/df.residual(fit)</code>
Residual standard error	$\sqrt{\text{MSE}}$	<code>summary(fit)\$sigma</code>
R^2	$1 - \text{SSE}/\text{SST}$	<code>summary(fit)\$r.squared</code>
Coefficient table	estimate, SE, t , p	<code>coef(summary(fit))</code>
Confidence intervals	$\hat{\beta}_j \pm t^* \text{SE}$	<code>confint(fit)</code>
Prediction	$\hat{y}(\mathbf{x}_0)$	<code>predict(fit, newdata=...)</code>
Nested-model test	partial F -test	<code>anova(reduced, full)</code>
Diagnostic plots	assumption checks	<code>plot(fit)</code>

3.1 Matrix operations in R

The following commands translate the matrix least-squares formula directly:

Mathematics	R	Purpose
$\mathbf{X}^T$	<code>t(X)</code>	transpose
$\mathbf{X}^T \mathbf{X}$	<code>t(X) %*% X</code>	matrix multiplication
$(\mathbf{X}^T \mathbf{X})^{-1}$	<code>solve(t(X) %*% X)</code>	inverse
$\hat{\beta}$	<code>solve(t(X) %*% X, t(X) %*% Y)</code>	solve normal equations

Use `%*%` for matrix multiplication. Ordinary `*` performs element-by-element multiplication. For numerical work, `solve(A, b)` is preferable to `solve(A) %*% b` because it solves $\mathbf{Ax} = \mathbf{b}$ without explicitly forming the inverse.

3.2 How to read `summary(lm(...))`

The coefficient table contains four columns:

1. **Estimate:** the fitted coefficient $\hat{\beta}_j$.
2. **Std. Error:** estimated standard deviation of $\hat{\beta}_j$ across repeated samples.
3. **t value:** $t = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$ for testing $H_0 : \beta_j = 0$.
4. **Pr(> |t|):** the two-sided p -value.

The last few lines give the residual standard error, R^2 , adjusted R^2 , and overall F -test. A small coefficient p -value concerns one predictor conditional on the others; the overall F -test concerns all slopes together.

4 Exercises

Start every exercise with:

```
data(mtcars)
cars <- mtcars
```

Exercise 1: Inspect the data. Use `head()`, `dim()`, `names()`, and `str()` to inspect `cars`.

- How many observations and variables are present?
- Are `mpg`, `wt`, and `hp` stored as numerical variables?
- What does one row represent?

Exercise 2: Summaries and the simple-regression formula. Let $X = \text{wt}$ and $Y = \text{mpg}$.

- Calculate $\bar{x}$, $\bar{y}$, s_x , and s_y with R.
- Calculate

$$S_{xx} = \sum (x_i - \bar{x})^2, \quad S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}).$$

- Use $\hat{\beta}_1 = S_{xy}/S_{xx}$ and $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1\bar{x}$.

Syntax hint: multiplication is `*`, powers use `^`, and `sum()` adds a vector.

Exercise 3: Simple linear regression. Fit $\text{mpg} \sim \text{wt}$ using `lm()`.

- Extract the coefficients with `coef()`.
- Write the fitted equation.
- Interpret the slope, remembering that `wt` is measured in thousands of pounds.
- Confirm that the coefficients agree with Exercise 2.

Exercise 4: Multiple linear regression. Fit $\text{mpg} \sim \text{wt} + \text{hp}$.

- Write the fitted equation.
- Interpret the `wt` coefficient while holding horsepower fixed.
- Interpret the `hp` coefficient while holding weight fixed.
- Compare the `wt` coefficient with the simple-regression value. Why might it change?

Exercise 5: Fitted values and residual identities. For the model in Exercise 4, store

```
y_hat <- fitted(full_model)
e <- resid(full_model)
```

Then verify numerically that

$$y_i = \hat{y}_i + e_i, \quad \sum e_i = 0, \quad \sum \text{wt}_i e_i = 0, \quad \sum \text{hp}_i e_i = 0.$$

Explain why R may display numbers such as 10^{-13} instead of exact zero.

Exercise 6: Matrix least squares. Use

```
X <- model.matrix(full_model)
Y <- matrix(cars$mpg, ncol = 1)
```

and calculate

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}.$$

Compare the result with `coef(full_model)`. Also calculate `t(X) %*% e`. What does the result mean?

Exercise 7: SSE, MSE, and R^2 . For the two-predictor model, manually calculate

$$\text{SSE} = \sum e_i^2, \quad \text{MSE} = \frac{\text{SSE}}{n - p - 1}, \quad R^2 = 1 - \frac{\text{SSE}}{\text{SST}}.$$

Use $n = 32$ and $p = 2$. Compare your results with `summary(full_model)`.

Exercise 8: A coefficient test and confidence interval. For the `hp` coefficient, test

$$H_0 : \beta_{\text{hp}} = 0 \quad \text{against} \quad H_1 : \beta_{\text{hp}} \neq 0.$$

- Extract its estimate and standard error.
- Calculate the t statistic manually.
- Calculate the two-sided p -value using `pt()`.
- Obtain a 95% confidence interval using `confint()`.
- State the conclusion at $\alpha = 0.05$.

Exercise 9: Confidence versus prediction intervals. For a car with `wt = 3` and `hp = 110`, calculate:

- the predicted fuel efficiency;
- a 95% confidence interval for the mean fuel efficiency;
- a 95% prediction interval for one new car.

Explain why the prediction interval is wider.

Exercise 10: Nested models and diagnostics. Let the reduced model be `mpg ~ wt` and the full model be `mpg ~ wt + hp`.

- Use `anova(reduced_model, full_model)` to test whether adding `hp` significantly improves the model.
- Verify that this partial F statistic equals the square of the `hp` t statistic from Exercise 8.
- Run `plot(full_model)`. State what each of the four standard plots is designed to check.
- Identify the observations with the largest absolute standardized residual and largest Cook's distance. Should they be deleted automatically?

5 Worked solutions

Solution 1.

```
head(cars)
dim(cars)
names(cars)
str(cars)
```

There are 32 observations and 11 variables. The three variables used here are stored as numerical variables. Each row represents one car model. $\square$

Solution 2.

```
x <- cars$wt
y <- cars$mpg
x_bar <- mean(x)
y_bar <- mean(y)
s_x <- sd(x)
s_y <- sd(y)
c(x_bar=x_bar, y_bar=y_bar, s_x=s_x, s_y=s_y)
```

The results are

$$\bar{x} = 3.21725, \quad \bar{y} = 20.090625, \quad s_x = 0.978457, \quad s_y = 6.026948.$$

Next,

```
Sxx <- sum((x - x_bar)^2)
Sxy <- sum((x - x_bar) * (y - y_bar))
b1 <- Sxy / Sxx
b0 <- y_bar - b1 * x_bar
c(Sxx=Sxx, Sxy=Sxy, b0=b0, b1=b1)
```

This gives

$$S_{xx} = 29.678748, \quad S_{xy} = -158.617225, \\ \hat{\beta}_0 = 37.285126, \quad \hat{\beta}_1 = -5.344472.$$

$\square$

Solution 3.

```
simple_model <- lm(mpg ~ wt, data = cars)
coef(simple_model)
```

The output is

```
(Intercept)      wt
 37.285126    -5.344472
```

Therefore

$$\widehat{\text{mpg}} = 37.2851 - 5.3445 \text{ wt}.$$

A 1000-pound increase in weight is associated with an estimated decrease of about 5.34 mpg. This is an association, not automatically a causal effect. The coefficients agree with the manual least-squares calculation. $\square$

Solution 4.

```
full_model <- lm(mpg ~ wt + hp, data = cars)
coef(full_model)
```

R returns

```
(Intercept)      wt      hp
37.22727012 -3.87783074 -0.03177295
```

Thus

$$\widehat{\text{mpg}} = 37.2273 - 3.8778 \text{ wt} - 0.031773 \text{ hp}.$$

Holding horsepower fixed, an additional 1000 pounds is associated with about 3.88 fewer mpg. Holding weight fixed, one additional horsepower is associated with about 0.0318 fewer mpg. The weight coefficient changes from -5.3445 to -3.8778 because weight and horsepower contain overlapping information; a multiple-regression coefficient isolates the contribution of one predictor conditional on the other. $\square$

Solution 5.

```
y_hat <- fitted(full_model)
e <- resid(full_model)
max(abs(cars$mpg - (y_hat + e)))
sum(e)
sum(cars$wt * e)
sum(cars$hp * e)
```

The first result is exactly 0 in this run. The remaining results are approximately

$$3.44 \times 10^{-15}, \quad 4.47 \times 10^{-15}, \quad 2.86 \times 10^{-13}.$$

Mathematically, all three are zero. Computers store most decimal values with finite binary precision, so tiny rounding errors remain. They should be read as numerical zero, not as violations of the least-squares identities. $\square$

Solution 6.

```
X <- model.matrix(full_model)
Y <- matrix(cars$mpg, ncol = 1)
beta_matrix <- solve(t(X) %*% X, t(X) %*% Y)
beta_matrix
coef(full_model)
```

Both methods give

$$\hat{\beta} = \begin{pmatrix} 37.22727012 \\ -3.87783074 \\ -0.03177295 \end{pmatrix}.$$

The check

```
drop(t(X) %*% matrix(e, ncol = 1))
```

produces values on the order of 10^{-13} or smaller. Thus $\mathbf{X}^T \mathbf{e} = \mathbf{0}$: residuals are orthogonal to the intercept column and both predictor columns. This condition is the matrix form of the normal equations. $\square$

Solution 7.

```
n <- nrow(cars)
p <- 2
SSE <- sum(e^2)
MSE <- SSE / (n - p - 1)
SST <- sum((cars$mpg - mean(cars$mpg))^2)
R2 <- 1 - SSE / SST
c(SSE=SSE, MSE=MSE, residual_SE=sqrt(MSE), R2=R2)
```

The results are

$$\begin{aligned} \text{SSE} &= 195.047755, & \text{MSE} &= 6.725785, \\ \sqrt{\text{MSE}} &= 2.593412, & R^2 &= 0.826786. \end{aligned}$$

The error degrees of freedom are $32 - 2 - 1 = 29$. The residual standard error and R^2 agree with `summary(full_model)`. The adjusted value is $R^2_{\text{adj}} = 0.814840$. $\square$

Solution 8.

```
tab <- coef(summary(full_model))
t_hp <- tab["hp", "Estimate"] / tab["hp", "Std. Error"]
df_error <- df.residual(full_model)
p_hp <- 2 * pt(abs(t_hp), df=df_error, lower.tail=FALSE)
c(t_hp=t_hp, df=df_error, p_value=p_hp)
confint(full_model, "hp", level=0.95)
```

For horsepower,

$$\hat{\beta}_{\text{hp}} = -0.03177295, \quad \text{SE}(\hat{\beta}_{\text{hp}}) = 0.00902971.$$

Therefore

$$t = \frac{-0.03177295}{0.00902971} = -3.518712, \quad df = 29,$$

and the two-sided p -value is 0.001451. The 95% confidence interval is

$$(-0.050241, -0.013305).$$

Because $p < 0.05$, reject H_0 . There is evidence that horsepower is associated with mpg after controlling for weight. The confidence interval supports the same conclusion because it excludes zero. $\square$

Solution 9.

```
new_car <- data.frame(wt=3, hp=110)
predict(full_model, newdata=new_car,
        interval="confidence", level=0.95)
predict(full_model, newdata=new_car,
        interval="prediction", level=0.95)
```

The predicted value is 22.09875 mpg. The 95% confidence interval for the mean response is

$$(21.01879, 23.17871),$$

while the 95% prediction interval for one new car is

$$(16.68580, 27.51170).$$

The prediction interval is wider because it includes both uncertainty in the estimated mean and the natural car-to-car error ε_{new} . $\square$

Solution 10.


```
reduced_model <- lm(mpg ~ wt, data=cars)
anova(reduced_model, full_model)
```

The comparison reports

$$F = 12.381, \quad p = 0.001451.$$

Adding horsepower significantly reduces SSE, so the full model is preferred by this test. Also

$$(-3.518712)^2 = 12.381,$$

confirming $F = t^2$ when one coefficient is added.

Run the standard plots with

```
par(mfrow=c(2, 2))
plot(full_model)
par(mfrow=c(1, 1))
```

Their main purposes are:

1. **Residuals vs Fitted:** check linearity and look for changing spread.
2. **Normal Q-Q:** check whether residuals are approximately normal.
3. **Scale-Location:** check constant error variance.
4. **Residuals vs Leverage:** find potentially influential cases.

Observation 20 (Toyota Corolla) has the largest absolute standardized residual, and observation 17 (Chrysler Imperial) has the largest Cook's distance. A flag means “investigate,” not “delete.” Removal requires a data-quality or scientific justification. $\square$

One-page R regression checklist

```
# 1. Inspect
head(d); str(d); summary(d)

# 2. Fit and read
fit <- lm(y ~ x1 + x2, data=d)
summary(fit)
coef(fit)
confint(fit)

# 3. Extract
y_hat <- fitted(fit)
e <- resid(fit)
X <- model.matrix(fit)

# 4. Verify the math
sum(e) # approximately 0
t(X) %*% e # approximately zero vector
SSE <- sum(e^2)
MSE <- SSE / df.residual(fit)
R2 <- 1 - SSE / sum((d$y - mean(d$y))^2)

# 5. Predict
new_case <- data.frame(x1=..., x2=...)
predict(fit, new_case, interval="confidence")
predict(fit, new_case, interval="prediction")

# 6. Compare and diagnose
anova(reduced_fit, full_fit)
par(mfrow=c(2, 2)); plot(fit)
```

Final memory rule:

formula $\rightarrow$ <code>lm()</code> $\rightarrow$ <code>summary()</code> $\rightarrow$ residual checks
--

Common mistakes

- Use `==` to test equality; use `<-` to assign a value.
- Use `%*%` for matrix multiplication, not `*`.
- Put new predictor values in a data frame with exactly the original variable names before calling `predict()`.
- In $n - p - 1$, p counts predictors but not the intercept.
- Do not interpret a multiple-regression slope without saying “holding the other predictors fixed.”
- A small p -value does not measure practical importance or prove causation.
- Diagnostic flags do not automatically justify deleting observations.